

自动编码器

学习目标

自动编码器有什么用

什么是自动编码器（Autoencoder）

定义

原理作用案例

类别

Keras快速搭建普通自编码器——基于Mnist手写数字

自编码器效果

流程

代码编写

基于Mnist手写数字-深度自编码器

基于Mnist手写数字——卷积自编码器

基于Mnist手写数字-降噪自编码器

学习目标

- 目标
 - 了解自动编码器作用
 - 说明自动编码器的结构
- 应用
 - 使用自动编码器对Mnist手写数字进行数据降噪处理

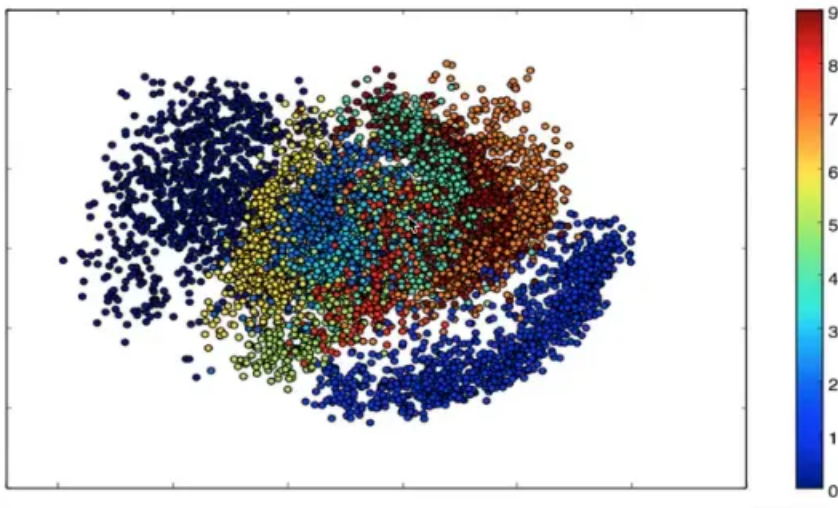
自动编码器有什么用

自编码器的应用主要有两个方面

- 数据去噪

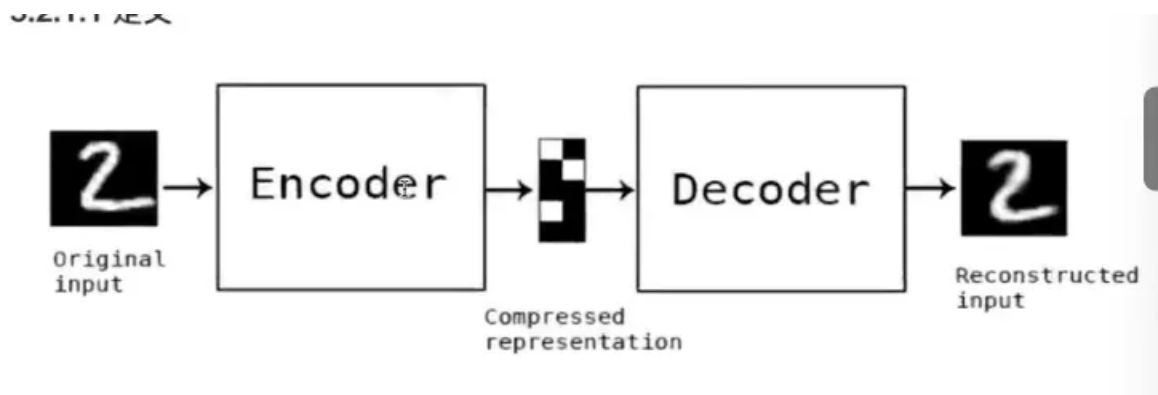


- 进行可视化降维
 - 自编码可以学习到比PCA等技术更好的数据投影



什么是自动编码器 (Autoencoder)

定义

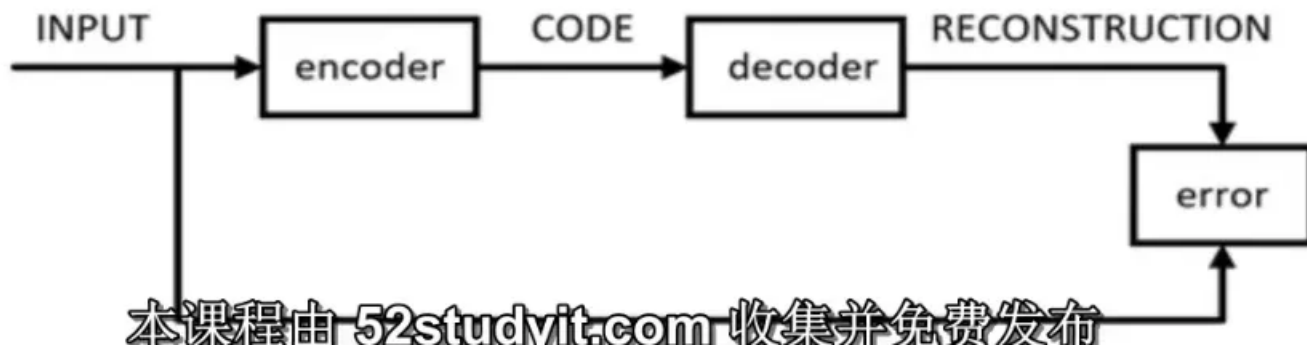


自动编码器是一种数据的压缩算法，一种使用神经网络学习数据值编码的无监督方式

原理作用案例

搭建一个自动编码器需要完成下面三样工作：

- 搭建编码器
- 搭建解码器
- 设定一个损失函数，用以衡量由于压缩而损失掉的信息。
 - 编码器和解码器一般都是参数化的方程，并关于损失函数可导，通常情况是使用神经网络。

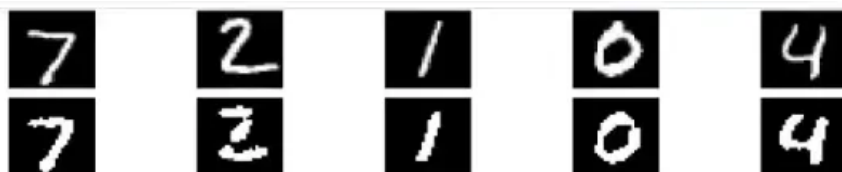


类别

- 普通自编码器
 - 细解码网络使用全连接层
- 多层自编码器
- 卷积自编码器
 - 编解码使用卷积结构
- 正则化自编码器
 - 降噪自编码器

Keras快速搭建普通自编译器——基于Minst手写数字

自编译器效果



```

Train on 60000 samples, validate on 10000 samples
Epoch 1/50
 256/60000 [.....] - ETA: 44s - loss: 0.6957
1280/60000 [.....] - ETA: 11s - loss: 0.6867
2560/60000 [>.....] - ETA: 6s - loss: 0.6699
3584/60000 [>.....] - ETA: 5s - loss: 0.6493
...
...
...
55808/60000 [=====>...] - ETA: 0s - loss: 0.0925
57088/60000 [=====>..] - ETA: 0s - loss: 0.0925
58112/60000 [=====>.] - ETA: 0s - loss: 0.0925
59392/60000 [=====>.] - ETA: 0s - loss: 0.0925
60000/60000 [=====] - 3s 47us/step - loss: 0.0925 - val_loss: 0.0
  
```

流程

- 初始化自编码器结构
- 训练自编码器
 - 获取数据
 - 模型输入输出训练
- 显示自编码前后效果对比

代码编写

导入所需包

- 1、初始化自编码器结构

定义编码器：输出32个神经元，使用relu激活函数，(32这个值可以自己制定)

定义解码器：输出784个神经元，使用sigmoid函数，(784这个值是输出与原图片大小一致)

损失：

- 每个像素值的交叉熵损失（输出为sigmoid值(0,1)，输入图片要进行归一化(0,1)

```
from keras.layers import Input, Dense
from keras.layers import Conv2D, MaxPooling2D, UpSampling2D
from keras.models import Model
from keras.datasets import mnist
import numpy as np
import matplotlib.pyplot as plt

class AutoEncoder(object):
    """自动编码器"""
    def __init__(self):
        self.encoding_dim = 32
        self.decoding_dim = 784
        self.model = self.auto_encoder_model()

    def auto_encoder_model(self):
        """
        初始化自动编码器模型
        将编码器和解码器放在一起作为一个模型
        :return: auto_encoder
        """
        input_img = Input(shape=(784,))

        encoder = Dense(self.encoding_dim, activation='relu')(input_img)
        decoder = Dense(self.decoding_dim, activation='sigmoid')(encoder)
        auto_encoder = Model(inputs=input_img, outputs=decoder)
        auto_encoder.compile(optimizer='adam', loss='binary_crossentropy')
        return auto_encoder
```

```
# def auto_encoder_model(self):
#     """
#     初始化自动编码器模型
#     将编码器和解码器放在一起作为一个模型
#     :return: auto_encoder
#     """
#     input_img = Input(shape=(28, 28, 1))
#
#     x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
#     x = MaxPooling2D((2, 2), padding='same')(x)
#     x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
#     encoded = MaxPooling2D((2, 2), padding='same')(x)
#     print(encoded)
#
#     x = Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
#     x = UpSampling2D((2, 2))(x)
#     x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
#     x = UpSampling2D((2, 2))(x)
```

```
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x)
print(encoded)

x = Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
print(decoded)

auto_encoder = Model(input_img, decoded)
auto_encoder.compile(optimizer='adam', loss='binary_crossentropy')

return auto_encoder
```

- 2、训练流程
- 读取Mnist数据，并进行归一化处理以及形状修改
- 模型进行fit训练
 - 指定迭代次数
 - 指定每批次数据大小
 - 是否打乱数据
 - 验证集合

```
def train(self):
    """
    训练自编码器
    :param model: 编码器结构
    :return:
    """
    (x_train, _), (x_test, _) = mnist.load_data()

    # 进行归一化
    x_train = x_train.astype('float32') / 255.
    x_test = x_test.astype('float32') / 255.

    # 进行形状改变
    x_train = np.reshape(x_train, (len(x_train), np.prod(x_train.shape[1:])))
    x_test = np.reshape(x_test, (len(x_test), np.prod(x_test.shape[1:])))

    print(x_train.shape)
    print(x_test.shape)

    # 训练
    self.model.fit(x_train, x_train,
                   epochs=5,
                   batch_size=256,
                   shuffle=True,
                   validation_data=(x_test, x_test))
```

- 3、显示模型生成的图片与原始图片对比
 - 导入matplotlib包

```
def display(self):
    """
    显示前后效果对比
    :return:
    """
    (x_train, _), (x_test, _) = mnist.load_data()

    x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))

    x_test_noisy = x_test + np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)

    decoded_imgs = self.model.predict(x_test_noisy)

    plt.figure(figsize=(20, 4))
    # 显示5张结果
    n = 5
    for i in range(n):
        # 显示编码前结果
```

```
        # 显示5张结果
        n = 5
        for i in range(n):
            # 显示编码前结果
            ax = plt.subplot(2, n, i + 1)
            plt.imshow(x_test_noisy[i].reshape(28, 28))
            plt.gray()
            ax.get_xaxis().set_visible(False)
            ax.get_yaxis().set_visible(False)

            # 显示解码后结果
            ax = plt.subplot(2, n, i + n + 1)
            plt.imshow(decoded_imgs[i].reshape(28, 28))
            plt.gray()
            ax.get_xaxis().set_visible(False)
            ax.get_yaxis().set_visible(False)
        plt.show()
```

```
if __name__ == '__main__':
    ae = AutoEncoder()
    ae.train()
    ae.display()
```

基于Mnist手写数字-深度自编码器

将多个自编码进行重叠

```
input_img = Input(shape=(784,))
encoded = Dense(128, activation='relu')(input_img)
encoded = Dense(64, activation='relu')(encoded)
encoded = Dense(32, activation='relu')(encoded)

decoded = Dense(64, activation='relu')(encoded)
decoded = Dense(128, activation='relu')(decoded)
decoded = Dense(784, activation='sigmoid')(decoded)

auto_encoder = Model(input=input_img, output=decoded)
auto_encoder.compile(optimizer='adam', loss='binary_crossentropy')
```

我们可以替换原来的编码器进行测试

```
59392/60000 [=====>.] - ETA: 0s - loss: 0.0860
```

最后的损失会较之前同样的epoch迭代好一些

基于Mnist手写数字——卷积自编码器

- 卷积编解码结构设计

- 编码器

- Conv2D(32, (3, 3), activation='relu', padding='same')
 - MaxPooling2D((2, 2), padding='same')
 - Conv2D(32, (3, 3), activation='relu', padding='same')
 - MaxPooling2D((2, 2), padding='same')
 - 输出大小: Tensor("max_pooling2d_2/MaxPool:0", shape=(?, 7, 7, 32), dtype=float32)

- 解码器:反卷积过程

- Conv2D(32, (3, 3), activation='relu', padding='same')
 - UpSampling2D((2, 2))
 - Conv2D(32, (3, 3), activation='relu', padding='same')
 - UpSampling2D((2, 2))
 - Conv2D(1, (3, 3), activation='sigmoid', padding='same')
 - 输出大小: Tensor("conv2d_5/Sigmoid:0", shape=(?, 28, 28, 1), dtype=float32)

```
input_img = Input(shape=(28, 28, 1))
```

```
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x)
print(encoded)
```

```
x = Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
print(decoded)
```

```
auto_encoder = Model(input_img, decoded)
auto_encoder.compile(optimizer='adam', loss='binary_crossentropy')
```

由于修改了模型的输入输出数据形状，所以在训练的地方同样也需要修改（显示的时候数据输入也要修改）

基于Mnist手写数字-降噪自编码器

- 降噪自编码器效果



- 过程
 - 对原始数据添加噪音
 - 随机加上正态分布的噪音
 - `x_train np.random.normal(loc=0.0,scale=1.0,size=x_train.shape)`

```
# 添加噪音
x_train_noisy = x_train + np.random.normal(loc=0.0, scale=3.0, size=x_train.shape)
x_test_noisy = x_test + np.random.normal(loc=0.0, scale=3.0, size=x_test.shape)

# 重新进行限制每个像素值的大小在0~1之间
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

在进行显示的时候也要进行修改

```
# 获取数据改变形状, 增加噪点数据
(x_train, _), (x_test, _) = mnist.load_data()
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))
x_test_noisy = x_test + np.random.normal(loc=3.0, scale=10.0, size=x_test.shape)

# 预测结果
decoded_imgs = self.model.predict(x_test_noisy)

# 修改需要显示的图片变量
plt.imshow(x_test_noisy[i].reshape(28, 28))
```